

Bloom filter

16 September 2026 04:17 PM

Bloom Filter

1. What is a Bloom Filter?

A **Bloom Filter** is a space-efficient data structure used to check whether an item **may exist** in a set.

It gives only two possible answers:

NO → Definitely NOT present

YES → Probably present

So:

Bloom Filter can have False Positives, but not False Negatives.

Why use it?

Because sometimes we want to avoid an **expensive lookup**.

Example:

Request

↓

Bloom Filter

↓

Definitely NOT → Don't check DB

Probably YES → Check DB

This can save disk reads, cache lookups, or network calls.

2. How Does It Work?

A Bloom Filter contains:

- **Bit array** of size m
- **k hash functions**

Initially:

0000000000

All bits are 0.

Adding an Item

Suppose we add:

"apple"

Assume 3 hash functions give:

$h_1 \rightarrow 2$

$h_2 \rightarrow 5$

$h_3 \rightarrow 8$

Set those positions to 1:

0010010010

↑ ↑ ↑

2 5 8

The actual "apple" is **not stored**.

Only the bits are stored.

3. Checking an Item

Suppose we want to check "apple".

Run the same hash functions:

$h_1 \rightarrow 2$

$h_2 \rightarrow 5$

$h_3 \rightarrow 8$

Check those bits.

Case 1: Any bit is 0

$h_1 \rightarrow 2 \rightarrow 1$

$h_2 \rightarrow 5 \rightarrow 0 \leftarrow$

$h_3 \rightarrow 8 \rightarrow 1$

Answer:

DEFINITELY NOT PRESENT

Because if "apple" had been inserted, all its bits would have been set to 1.

Case 2: All bits are 1

h1 → 2 → 1

h2 → 5 → 1

h3 → 8 → 1

Answer:

PROBABLY PRESENT

It **might** be present.

Why?

Because some **other items** may have set those same bits to 1.

4. False Positive

This is the most important concept.

Suppose:

apple → bits 2, 5, 8

banana → bits 1, 4, 5

Now check:

mango

Suppose mango hashes to:

2, 5, 8

All those bits are already 1.

Bloom Filter says:

Probably Present

But maybe mango was **never inserted**.

That's a:

False Positive

Important

Bloom Filter:

False Positive → YES when actually NO

False Negative → NO when actually YES **✗**

There should be **no false negative for correctly added items**.

5. Real-Life Example — URL Deduplication

Imagine a web crawler.

It has already visited:

google.com

youtube.com

amazon.com

...

There may be **millions/billions of URLs**.

Instead of storing every URL in a huge exact set, use a Bloom Filter as a quick first check.

New URL

↓

Bloom Filter

↓

Definitely NOT → Process URL

Probably YES → Maybe skip / verify

This saves memory and makes the first check very fast.

6. Important System Design Use Cases

1. LSM-Tree Databases

Examples:

- Cassandra
- RocksDB

A database may have many SSTable files.

Instead of checking every file:

Key

↓

Bloom Filter

↓

SSTable 1 → Definitely not

SSTable 2 → Definitely not

SSTable 3 → Maybe → Check it

This reduces unnecessary disk reads.

2. Cache

Suppose cache lookup is expensive.

Request



Bloom Filter



No → Skip cache lookup

Yes → Check cache

Useful when there are many cache misses.

3. Web Crawlers

Used to reduce duplicate URL processing.

But a false positive could cause a crawler to skip a URL that it hasn't actually visited, so exact verification may be needed when completeness matters.

7. Bloom Filter Sizing

Important variables:

Variable **Meaning**

n	Number of items expected
p	Acceptable false-positive rate
m	Number of bits
k	Number of hash functions

Example

For approximately:

100 million keys

1% false-positive rate

You need roughly:

9.6 bits / item

The source's table gives:

False Positive Rate	Bits / Item	Hash Functions
10%	~4.8	~3
1%	~9.6	~7
0.1%	~14.4	~10
0.01%	~19.2	~13

Remember

Lower false-positive rate → More memory required.

8. What Happens If We Add Too Many Items?

Bloom Filters need to be sized for expected capacity.

Initially:

000000000000000000

After many insertions:

111111111111111111

More bits become 1.

Therefore, more missing items may look like they are present.

So:

Overfilling → More False Positives

9. Why Can't Normal Bloom Filters Delete?

Suppose:

apple → bits 2, 5, 8

banana → bits 5, 7, 9

Bit 5 belongs to **both**.

If we delete apple and simply change its bits to 0:

bit 5 → 0

we accidentally affect banana too.

Therefore:

Standard Bloom Filter does not safely support deletion.

For deletion, use a **Counting Bloom Filter**.

10. Variants

Counting Bloom Filter

Instead of:

0 1 1 0

use counters:

0 2 1 0

Allows deletion by decrementing counters.

Tradeoff: more memory.

Scalable Bloom Filter

Adds new sub-filters as the data grows.

Useful when the final size isn't known in advance.

Cuckoo Filter

Uses fingerprints and buckets instead of shared bits.

A useful alternative when **deletion is common**.

11. Bloom Filter vs HashSet

	HashSet	Bloom Filter
Stores actual items	☒	☐
Membership check	Exact	Probabilistic
False positive	☐	☒
False negative	☐	☐
Memory	Higher	Lower
Delete	☒	☐ standard version